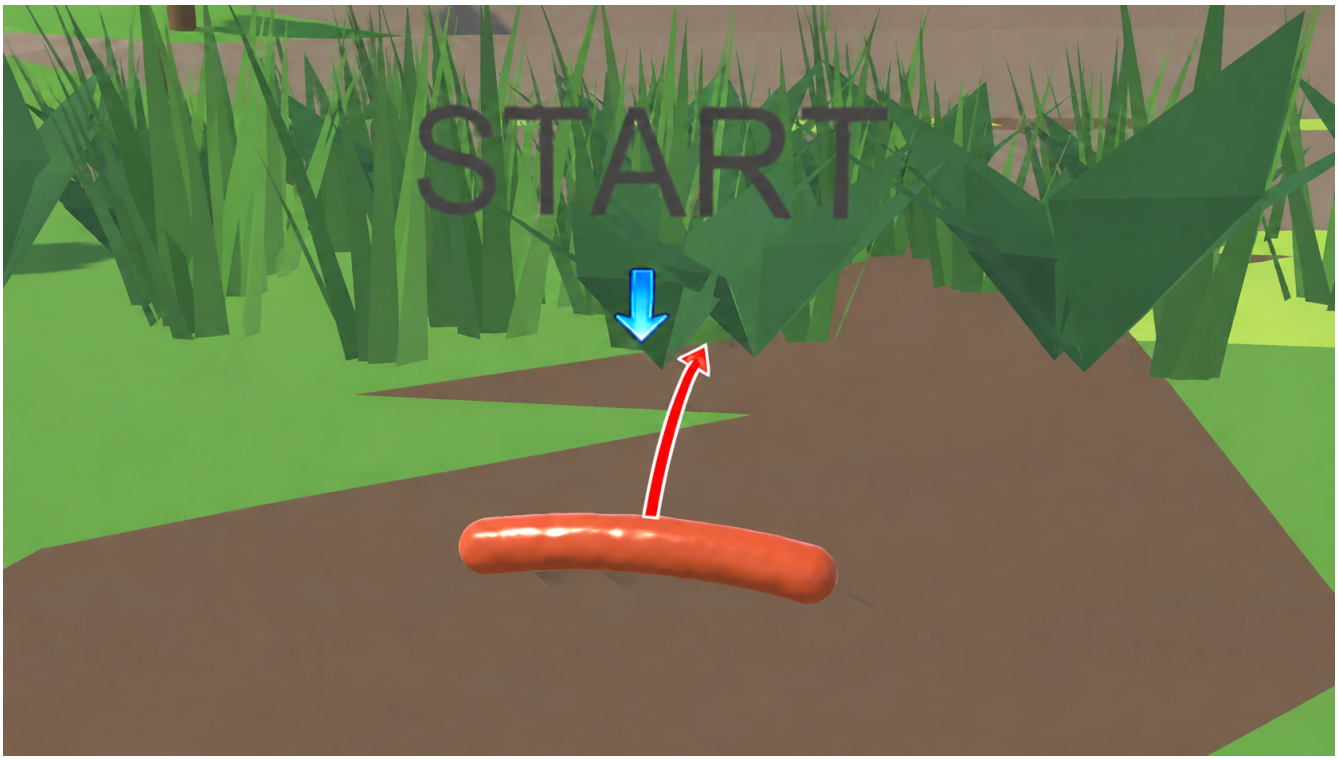
Beyond The Sausage

引っ張って飛ばし、空中で立て直す。VRの身体操作でコースを攻略する物理アクション。



開発中のゲーム画面：引っ張り入力と軌道ガイドで、飛ばす方向と強さを調整。

Made in Japan XR Project 最終選抜チーム
約50万円の開発支援を受けて制作中

私の担当：ゲーム企画全般・ソーセージの操作

ゲームの原案・企画全般と、ソーセージを操作する仕組みを担当しています。本資料では、**入力の座標変換、片手・両手の状態管理、発射・接地、軌道予測、振動フィードバック**を取り上げます。

チームの分担：ステージ・ギミック・レベルデザインの制作は別メンバーが担当。企画や操作の改善方針は、チーム・メンターとの議論とプレイ評価を踏まえて更新しています。

開発環境：Unity 6000.2.7f2 / C# / Unity Physics / UnityEngine.XR
対象：Meta Quest向けVR。開発中の実装を紹介する資料です。

対象コード：sausage-for-sausage / bda3b1b（2026-10-23確認）

腕の動きを、狙った発射へ変える

担当：Trigger操作 / 座標変換 / 可動域の補助 / 力点と発射速度

課題：狙う操作と、引っ張る動作が競合する

初期のRayを当てて掴む方式では、腕の位置が制約され、空中の対象を捉え直しにくいという課題がありました。現在は**Rayの命中を条件にせず、Triggerから操作を開始**します。操作の理解を簡単にし、攻略の工夫に集中できる方向へ見直しました。

工夫：Rigの移動と、手の移動を切り分ける

手のワールド座標をそのまま使うと、視点を動かした分まで引っ張り量に入ります。現在のRig内で手の位置差を求め、掴み始めの変換行列でワールド方向へ戻すことで、**Rig移動による入力の上乗せを抑えます。**

```
Vector3 current = pull.ReferenceSpace != null
    ? pull.ReferenceSpace.InverseTransformPoint(
        pull.Controller.position) : pull.Controller.position;
Vector3 input = pull.ReferenceToWorldAtGrab.MultiplyVector(
    current - pull.StartControllerPosition);
```

GetLaunchVector() の抜粋。表示用に改行を調整しています。

工夫：入力距離・力点・速度上限を分ける

基本35cmの腕の移動で最大入力になる設定と、前後方向の可動域補助を用意。片手は左右それぞれの端、両手は中心を力点に使います。両手では入力ベクトルの平均に倍率を掛け、最後に速度上限を適用します。

設計の狙い：腕の移動量とゲーム内の飛距離を別々に調整できるようにする。体格差への配慮を進めつつ、ステージ制作側へ操作基準を共有する。

実装参照：SausagePullLauncher.cs / BeginPull(), GetLaunchVector(), GetForcePoint()

両手入力と空中操作を、状態で管理

担当：入力の集約 / 解除・再受付 / 接地による回数回復

左右の処理順に左右されない入力解決

左右ControllerのUpdateで入力を収集し、LauncherのLateUpdateでまとめて処理します。両手で引いている場合も**最初のReleaseで一度だけ発射**し、両手が離れた有効な入力を確認するまで再受付をロックします。追跡や入力が失われた場合は発射ではなくキャンセルとして扱います。

待機 → 片手 / 両手

左右入力を集約して開始

Release → 発射ロック

初回解除で発射・回数を1消費

両手OFF → 待機

有効な解除入力で再受付

「何かに触れた」だけでは回数を戻さない

接触面の法線と重力の逆方向の内積から、床・坂と壁・天井を区別します。現設定では45度以内の支持面を回復対象とし、引っ張り中は回復させません。離陸できなかった場合の回復も、発射後の待ち時間と接触の継続時間を条件にします。

```
Vector3 up = Physics.gravity.sqrMagnitude > 0.000001f
    ? -Physics.gravity.normalized : Vector3.up;
return Vector3.Dot(normal, up) >=
    Mathf.Cos(MaximumRecoverySlopeAngle * Mathf.Deg2Rad)
    - 0.00001f;
```

実装参照：SausagePullLauncher.cs / ProcessTriggerInputs(), Release(), IsRecoverySurface()

発射と予測で、同じ速度を使う

担当：発射軌道の計算 / 衝突の打ち切り / ガイド表示率

予測だけ別の計算にならない構成

実発射と予測表示の両方からGetPreviewVelocity()を使い、手数・パワー倍率・速度上限をそろえます。実発射時は引っ張り中の下降速度を消去し、同じ最終速度をAddForceAtPosition()へ渡します。

```
Vector3 velocity = GetPreviewVelocity(launchVector);
body.linearVelocity = Vector3.zero;
body.AddForceAtPosition(velocity, GetCurrentForcePoint(),
    ForceMode.VelocityChange);
```

最初の衝突まで計算してから、見せる長さを決める

重力・linearDamping・fixedDeltaTimeを使って経路を数値積分。区間ごとにRaycastし、自己ColliderやTriggerを除外して最も近い衝突で終了します。検索バッファが満杯なら拡張して再取得し、結果の順序に依存せず最短距離を選びます。

その経路の全長から表示する割合を決め、現在の初期値は50%です。「**予測をどこまで見せるか**」と「**どれだけ強く飛ぶか**」**を分離**し、手助けと試行錯誤のバランスを調整できます。衝突を見つけられなければ、途中で打ち切った経路を100%として表示しません。

近似の限界：点の経路に対するRaycastであり、回転する本体の体積、バウンド、動く足場の未来位置は再現しません。ガイドは補助であり、正確な着地点の保証ではありません。

今後の確認：足場の端・低い天井・片手による回転で、表示と実際の接触の差を実機比較する。予測の負荷もQuest上で計測する。

実装参照：SausagePullLauncher.cs / Launch()、TrajectoryArrow.cs / BuildLandingPath()

手触りと調整のしやすさを支える

担当：引っ張り・発射の振動 / 操作パラメータの管理

視覚に加えて、力の溜まり具合を手に伝える

予測される発射速度を最大速度で割った値を、引っ張りの強さとして使います。その値に応じて振動の振幅と間隔を補間し、実際に操作を受け付けた手だけに通知。発射時は引っ張りの振動を止め、発射用の振動へ切り替えます。

```
float interval = Mathf.Max(0.02f,
    Mathf.Lerp(weakPullInterval, strongPullInterval, power));
float amplitude = Mathf.Clamp01(
    Mathf.Lerp(weakPullAmplitude, strongPullAmplitude, power));
device.SendHapticImpulse(0, amplitude,
    Mathf.Min(pullPulseDuration, interval * 0.5f));
```

UpdatePullHaptics() の関連行抜粋。条件判定などを省略しています。

操作の設定を、ScriptableObjectへまとめる

SausagePhysicsConfigに腕の移動量、発射倍率・上限、回数回復条件、Rigidbodyの設定を集約。Awakeで実際のRigidbodyへ値を反映し、予測側はその値を参照します。コードを書き換えずに操作感を調整でき、ステージ担当との基準合わせにも使えます。

確認した設定アセットの値（調整中）

腕の基本移動量 0.35m / 発射速度上限 11.25m/s

引っ張り中の重力 0.2倍 / 回復面の最大傾斜 45度

実装済み：引っ張り強度に応じた振動と、発射時の振動。
今後の検討：接触の強さに応じた振動や効果音。実機での感触評価を継続します。

実装参照：ControllerRaySausageInteractor.cs、SausagePhysicsConfig.cs / .asset

企画の意図を、操作と検証へつなぐ

企画全般を担当。チーム・メンターの意見も取り入れ、実装と構想を分けて管理。

方針：操作は分かりやすく、習熟の余地は広く

飛ばす方向・強さ・空中での立て直しを試しながら、コースの攻略を考えるゲームを目指しています。操作の難しさとステージの難しさを混同せず、**「なぜ失敗したか」を理解して再挑戦できる設計**を重視しています。

改善の流れ：遊びにくさを、実装の課題へ落とす

第1回議事録では、Ray照準と引っ張りが競合する問題を整理。第3回では、腕の短い人でも扱える入力距離や、空中操作中の落下を調整する方針を確認しました。現在の実装ではRay不要のTrigger操作、可動域の補助、緩やかな落下へ反映されています。

第4回では、私からギミックごとの難易度がばらつく懸念を提示。序盤で学ぶ操作を先に明確にし、ギミックの目的・導入順をステージ担当と共有する方向へ整理しました。

現在の実装と、これからの取り組み

コードで確認できるもの：片手・両手入力、空中での再発射、接地回復、軌道ガイド、引っ張り・発射の振動。チーム制作の山・塔・メニューなども手元の開発版へ統合されています。

次に確かめたいこと：初心者の操作理解、ガイド50%の分かりやすさ、ステージ配置と操作基準の整合、Quest実機での快適さと負荷。効果音などの手触り改善も検討中です。

将来構想：習得に合わせた連続コースの拡充。マルチプレイやランキングはシングルプレイ完成後の候補で、実装済み機能としては扱いません。

根拠：第1〜4回メンタリング議事録（2026/9/21〜10/13）、開発計画、最新コード。 本資料作成時はコード・記録を照合。Quest実機試験は再実施していません。